

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Kiss Norbert

2026

**Szegedi Tudományegyetem
Informatikai Intézet**

**InstaCover:
AI könyvborító készítő webalkalmazás**

Szakdolgozat

Készítette:

Kiss Norbert

Programtervező

informatikus szakos hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi adjunktus

Szeged
2026

Tartalomjegyzék

TARTALMI ÖSSZEFOGLALÓ.....	6
1. BEVEZETÉS.....	7
1.1. A probléma.....	7
1.2. A megoldás.....	7
1.3. A dolgozat felépítése.....	8
2. PIACKUTATÁS.....	9
2.1. Célcsoport.....	9
2.2. Versenytárs-elemzés.....	9
2.3. Piaci rés.....	10
2.4. Értékaajánlat.....	10
3. FELHASZNÁLT TECHNOLOGIÁK.....	11
3.1. Frontend technológiák.....	11
3.2. Backend technológiák.....	11
3.3. Külső szolgáltatások.....	12
3.4. Infrastruktúra.....	12
4. ARCHITEKTÚRA.....	14
4.1. Rendszer áttekintés.....	14
4.2. Frontend architektúra.....	14
4.3. Backend architektúra.....	15
4.4. Adatmodell.....	15
4.5. Deployment architektúra.....	16
4.6. Adatfolyam.....	16
5. AI GENERÁLÁSI PIPELINE.....	18
5.1. Pipeline architektúra áttekintése.....	18
5.2. Standard pipeline.....	19
5.3. Sablon pipeline.....	20
5.4. Prompt engineering.....	21
5.5. Képgenerálási integráció.....	23

5.6. Kredit-integráció a pipeline-ban.....	24
6. STÍLUSREFERENCIA RENDSZER.....	25
6.1. Stíluselemzés feltöltéskor.....	25
6.2. Szövegdetektálás és szelekció.....	26
6.3. Referencia variánsok.....	26
6.4. Referencia módok.....	27
6.5. Text blending módok.....	28
7. VALÓS IDEJŰ KOMMUNIKÁCIÓ.....	30
7.1. Kommunikációs architektúra.....	30
7.2. Eseménymodell.....	30
7.3. Háttérfeladat-kezelés.....	31
7.4. Hibakezelés és állapot-konzisztencia.....	32
7.5. Reconnect és állapot-visszaállítás.....	32
7.6. Stale detection.....	33
7.7. Frontend állapotgép.....	33
7.8. Rate limiting.....	34
8. BIZTONSÁG ÉS KREDITRENDSZER.....	35
8.1. Autentikáció.....	35
8.2. Meghívó rendszer.....	35
8.3. Row-Level Security.....	36
8.4. Input validáció és prompt injection.....	36
8.5. Rate limiting.....	37
8.6. Kreditrendszer.....	37
9. TESZTELÉS ÉS CI/CD.....	40
9.1. Tesztelési stratégia.....	40
9.2. Mock infrastruktúra.....	40
9.3. Teszt típusok.....	41
9.4. CI/CD pipeline.....	41
10. MESTERSÉGES INTELLIGENCIA HASZNÁLATA A FEJLESZTÉS SORÁN....	42

10.1. Felhasznált eszközök.....	42
10.2. Felhasználási területek.....	42
10.3. Validáció és kritikus szemlélet.....	43
10.4. Ahol nem használtam MI-t.....	44
10.5. Tanulságok.....	44
11. ÖSSZEFOGLALÁS.....	46
11.1. Eredmények.....	46
11.2. Tapasztalatok.....	46
11.3. Továbbfejlesztési lehetőségek.....	47
11.4. Záró gondolatok.....	47
IRODALOMJEGYZÉK.....	48
NYILATKOZAT.....	49

Tartalmi összefoglaló

Szakedolgozatomban az InstaCover webalkalmazást mutatom be, amely mesterséges intelligencia segítségével automatizálja a könyvborító-készítés teljes folyamatát indie szerzők számára. Az alkalmazás nagy nyelvi modelleket (LLM) kombinál kreatív promptgeneráláshoz, korszerű képgenerálási technológiát a vizuális elemek elkészítéséhez, és automatikus tipográfiát a szöveges réteg hozzáadásához.

A rendszer három generálási pipeline-t implementál: a standard pipeline könyv adatokból generál borítót, a stílusreferencia pipeline feltöltött borítók vizuális stílusát reprodukálja AI-alapú elemzés alapján, a sablon pipeline pedig HTML/CSS sablonon alapuló pixelpontos tipográfiai kontrollt biztosít szerver oldali Chromium rendereléssel.

A technikai megvalósítás React frontend-re, Flask backend-re és Supabase szolgáltatásokra (PostgreSQL, autentikáció, fájl tárolás) épül. A generálási folyamat valós időben követhető WebSocket-en keresztül. A biztonságot Row-Level Security policy-k, prompt injection detektálás, input validáció és atomikus kreditműveletek biztosítják.

A dolgozat bemutatja a többlépcsős pipeline architektúra tervezését, a prompttervezés módszertanát, az aszinkron képgenerálási integráció megvalósítását, valamint a stílusreferencia rendszer működését (képdekompozíció, variáns-cache-elés, háromféle blending mód). A 204 automatizált backend teszt és a CI/CD pipeline biztosítja a kód minőségét. A dolgozat kitér a fejlesztés során alkalmazott MI-eszközök használatának tapasztalataira is.

1. Bevezetés

A self-publishing iparág az elmúlt évtizedben robbanásszerű növekedésen ment keresztül[18]. Az Amazon Kindle Direct Publishing és hasonló platformok lehetővé tették, hogy bárki publikáljon könyvet hagyományos kiadó nélkül. Ezzel együtt megnőtt az igény professzionális megjelenésű könyvborítók iránt, mivel a borító a vásárlói döntés egyik legfontosabb vizuális tényezője.

Az önálló szerzők (indie authors) számára a borítókészítés hagyományosan két úton érhető el: grafikus megbízása (20-200 dollár/borító, napok-hetek átfutással) vagy saját készítés általános célú tervezőeszközökkel (időigényes, gyakran amatőr eredménnyel). Egyik megoldás sem ideális költségérzékeny, több könyvet publikáló szerzők számára.

Szakedolgozatomban egy webalkalmazást mutatok be, amely mesterséges intelligencia segítségével automatizálja a könyvborító-készítés teljes folyamatát. Az **InstaCover** nevű alkalmazás nagy nyelvi modelleket (LLM) kombinál kreatív promptgeneráláshoz, korszerű képgenerálási technológiát a vizuális elemek elkészítéséhez, és automatikus tipográfiát a szöveges réteg hozzáadásához.

1.1. A probléma

A meglévő megoldások nem képesek egyszerre teljesíteni az alábbi elvárásokat:

- Teljes automatizáció a leírástól a kész borítóig, manuális tervezés nélkül
- Vizuális konzisztencia sorozat-borítók között
- Másodpercek-percek alatti kiszolgálás
- Néhány centes egységköltség borítónként

1.2. A megoldás

Az InstaCover három generálási pipeline-t implementál, amelyek eltérő felhasználói igényeket szolgálnak ki:

1. A standard pipeline a felhasználó könyvadatai (cím, szerző, műfaj, borítóötletek) alapján generál teljes borítót
2. A stílusreferencia pipeline egy feltöltött borítókép vizuális stílusát elemzi AI-val és reprodukálja

3. A sablon pipeline HTML/CSS sablonon alapuló, pixelpontos tipográfiai kontrollt biztosít

Az alkalmazás valós idejű visszajelzést ad a generálási folyamatról WebSocket-en keresztül, kreditrendszerrel biztosítja a költségkontrollt, és zárt béta meghívó-rendszerrel kezeli a hozzáférést.

1.3. A dolgozat felépítése

A dolgozat a következőképpen épül fel: a 2. fejezetben a piackutatás eredményeit mutatom be, a 3. fejezetben a felhasznált technológiákat. A 4. fejezet a rendszer architektúráját tárgyalja. Az 5. fejezet a mesterséges intelligencia alapú generálási pipeline technikai részleteit mutatja be, a 6. fejezet pedig a stílusreferencia rendszert. A 7. fejezet a valós idejű kommunikációt, a 8. fejezet a biztonságot és kreditrendszert, a 9. fejezet a tesztelést és CI/CD-t ismerteti. A 10. fejezetben az MI-eszközök használatáról adok önreflexiót, végül a 11. fejezetben összefoglalom az eredményeket és a továbbfejlesztési lehetőségeket.

2. Piackutatás

Az InstaCover fejlesztése előtt piackutatást végeztem, hogy megértsem a meglévő megoldásokat, azonosítsam a piaci rést, és validáljam a projekt értékajánlatát. Ebben a fejezetben összefoglalom a versenytárs-elemzés eredményeit és az önálló szerzők (indie authors) igényeinek feltárását.

2.1. Célcsoport

A self-publishing piac évről évre növekszik[18]: egyre több szerző publikál könyvet hagyományos kiadó nélkül, platformokon mint az Amazon Kindle Direct Publishing. Ezeknek a szerzőknek borítóra van szükségük, de jellemzően nem rendelkeznek grafikai ismeretekkel, és a költségérzékenység magas, különösen a kezdők körében.

A tipikus fájdalompontok (felhasználói interjúk alapján): - Egy grafikus 20-200 dollárt kér borítónként, ami több könyv esetén jelentős költség - A DIY eszközök (pl. Canva) sok időt igényelnek és a végeredmény amatőr hatású - A sorozat-borítók vizuális konzisztenciáját nehéz fenntartani eszközváltás nélkül

2.2. Versenytárs-elemzés

Három fő versenytársat vizsgáltam meg:

2.2.1. Canva AI Book Cover Generator[12]

A Canva egy általános célú grafikai tervezőeszköz, amely rendelkezik AI képgenerálási funkcióval (Magic Media). Erősségei a nagy sablonkönyvtár és az ismert márkanev. Gyengesége, hogy az AI jellemzően csak a háttérképet generálja: a tipográfia, az elrendezés és a finomítás manuálisan szükséges. Az ingyenes verzió korlátozott, a Pro verzió havi előfizetést igényel.

2.2.2. TheBookCoverDesigner[14]

Premade borítókat árusít, amelyek megvásárlás után egyedi címmel és szerzőnévvel kerülnek kiszolgáltatásra. Az árak 50-159 dollár között mozognak. Nincs AI generálás és nincs testreszabhatóság a premade-eken túl; ha a felhasználó nem talál megfelelő borítót a katalógusban, más megoldást kell keresnie.

2.2.3. GetCovers[13]

Emberi grafikusok készítenek egyedi borítókat \$10-35 árkategóriában. Az előnye az emberi kreativitás és az “unlimited revisions” ígéret. A hátránya, hogy nem azonnali (napokat vesz igénybe) és a minőség változó.

2.3. Piaci rés

Szempont	Canva	TheBookCover Designer	GetCovers	InstaCover
AI generálás	Részleges	Nincs	Nincs	Teljes pipeline
Tipográfia	Manuális	Fix premade	Emberi	AI + sablon
Stílus-konzisztencia	Nincs	Nincs	Emberi	Stílusreferencia
Azonnali eredmény	Részleges	Igen (premade)	Nem	Igen
Költség	Havi díj	Per borító	Per borító	Kredit-alapú

Az InstaCover egyedülálló kombinációja: **teljes automatizáció** (a felhasználó leírja a könyvét és kap kész borítót), **stílus-reprodukció** (meglévő borító stílusának átvétele), és **azonnali kiszolgálás** (másodpercek alatt). Ezeket a képességeket egyetlen versenytárs sem kínálja együtt.

2.4. Értékajánlat

Az InstaCover értékajánlata: AI-alapú könyvborító-generátor indie szerzőknek, amely LLM prompttervezést, képgenerálást és automatikus tipográfiát kombinál egy többlépéses, valós idejű pipeline-ban. A felhasználónak elegendő megadnia a könyve alapadatait, és percek alatt professzionális minőségű borítót kap.

3. Felhasznált technológiák

Ebben a fejezetben röviden bemutatom a projekt során felhasznált technológiákat és könyvtárakat, kitérve arra, hogy milyen szerepet töltenek be a rendszerben.

3.1. Frontend technológiák

A React[1] egy komponens-alapú JavaScript UI könyvtár egyoldalas alkalmazások építéséhez. A választásom azért esett rá, mert kiterjedt ökoszisztémával rendelkezik (React Query, Framer Motion), és a komplex állapotgépek (generálási folyamat) kezeléséhez a Context API és hookök jól használhatók.

A TypeScript[2] a JavaScript típusos kiterjesztése, amely fordítási időben deríti fel a típushibákat. A generálási pipeline konfigurációs objektumai (referencia mód, blending mód, képarány) számos opcionális mezővel rendelkeznek, ahol a típusbiztonság megakadályozza a futásidejű hibákat.

A Vite[3] egy modern frontend build eszköz, amely natív ES modulokat használ fejlesztés közben. A hagyományos bundlerekhez (Webpack) képest nagyságrendekkel gyorsabb fejlesztői szerveret ad.

A TailwindCSS[7] egy utility-first CSS keretrendszer, amely előre definiált osztályokkal működik. Nem kell egyéni CSS fájlokat írni; a stílus közvetlenül a JSX-ben jelenik meg.

A TanStack Query[19] szerver-állapot kezelő könyvtár React-hez. Automatikus cache-elést, háttérfrissítést és loading/error állapotokat biztosít a REST API hívásokhoz.

A Framer Motion[20] deklaratív animációs könyvtár React-hez. A generálási progress panel átmeneteinek és az oldal-váltás animációknak a megvalósításához használom.

A Socket.IO kliens a Socket.IO[6] JavaScript kliens oldali könyvtára, amely WebSocket kapcsolatot épít fel a backend-del. Automatikus újracsatlakozást és esemény-alapú kommunikációt biztosít.

3.2. Backend technológiák

A Flask[4] egy lightweight Python web keretrendszer. Minimális beépített funkcionalitással rendelkezik, ami lehetővé teszi, hogy csak a szükséges kiegészítőket használjam. A Python ökoszisztéma (Pillow, requests) és a natív Socket.IO támogatás volt a fő motiváció.

A Flask-SocketIO Flask kiegészítő, amely WebSocket támogatást ad a Socket.IO protokollon keresztül. Szoba-alapú üzenetküldést és háttérfeladat-kezelést biztosít.

A Gunicorn egy production-ready Python WSGI/ASGI szerver. A fejlesztői Flask szerver helyett ez szolgálja ki az alkalmazást éles környezetben, worker folyamatokkal.

A Pillow (PIL) Python képfeldolgozó könyvtár. A szegélydetektáláshoz, képvágáshoz és a programmatikus alpha-kompozícióhoz (direct overlay blending) használom.

A Playwright böngésző-automatizáló könyvtár, amely headless Chromium-ot vezérel. A sablon pipeline utolsó lépésében HTML/CSS-ből PNG képet renderel pixelpontos tipográfiával.

A Flask-Limiter rate limiting kiegészítő, amely végpont-specifikus kérés-korlátokat állít be (pl. 30 kérés/perc).

3.3. Külső szolgáltatások

A Supabase[5] egy nyílt forráskódú Firebase-alternatíva, amely PostgreSQL adatbázist, autentikációt (JWT-vel), objektumtárolást és Row-Level Security-t biztosít egyetlen platformon. A választásom azért esett rá, mert az RPC függvények lehetővé teszik az atomikus kredit-műveleteket, és az RLS policy-k adatbázis-szintű adatszigetelést adnak.

Az OpenRouter[8] egy LLM API gateway, amely egyetlen felületen keresztül ad hozzáférést több LLM szolgáltató modelljéhez (Grok, Gemini). Előnye, hogy modellváltás egyetlen konfiguráció-módosítással lehetséges, kódváltoztatás nélkül.

A WaveSpeed AI[9] képgenerálási API, amely a ByteDance Seedream V4.5 modelljét üzemelteti. Aszinkron job-kezeléssel működik: a kérés beküldése után polling-gal kérdezhető az eredmény. A modell jó minőség/ár arányt kínál könyvborítók generálásához.

3.4. Infrastruktúra

A Docker[10] konténerizációs platform biztosítja, hogy az alkalmazás azonos környezetben fusson fejlesztés és éles üzem során. A Docker Compose két szolgáltatást definiál: a backend-et és a frontend-et.

A GitHub Actions[11] CI/CD szolgáltatás, amely minden push és pull request esetén automatikusan futtatja a teszteket, a TypeScript type check-et és a lintert.

Az Nginx HTTP szerver és reverse proxy. A frontend konténerben statikus fájlokat szolgál ki, a VPS szintjén pedig (Nginx Proxy Manager formájában) TLS terminációt és reverse proxy-t biztosít.

4. Architektúra

Az InstaCover egy háromrétegű webalkalmazás, amely egyoldalas frontend-et (SPA), Python backend-et és külső szolgáltatásokat kombinál. Ebben a fejezetben bemutatom a rendszer komponenseit, azok kapcsolatát, az adatmodellt és a deployment architektúrát.

4.1. Rendszer áttekintés

Az alkalmazás három fő rétegből áll:

- A frontend (React SPA) a felhasználói felületet szolgálja ki statikus fájlakként. A böngészőben fut, és REST API illetve WebSocket kapcsolaton keresztül kommunikál a backend-del.
- A backend (Flask API) REST végpontokat és WebSocket eseménykezelőket biztosít. Itt fut az üzleti logika: a generálási pipeline-ok, az LLM integráció, a képgenerálási hívások és a kredit-kezelés.
- A külső szolgáltatások réteg: Supabase[5] (adatbázis, autentikáció, fájl tárolás), OpenRouter[8] (LLM hozzáférés) és WaveSpeed AI[9] (képgenerálás).

A rétegek közötti kommunikáció kizárólag HTTPS-en zajlik. A frontend és backend között kétféle csatorna működik: REST API a CRUD műveletekhez (generálási előzmények, stílusreferenciák, sablonok kezelése) és WebSocket a valós idejű generálási folyamathoz.

4.2. Frontend architektúra

A frontend egy React[1] alapú egyoldalas alkalmazás TypeScript[2]-ben, Vite[3] build eszközzel. Az alkalmazás állapotkezelése három React Context-re épül:

- Az AuthContext a Supabase Auth integrációt kezeli: session menedzsment, token frissítés, felhasználói profil
- A GenerationContext a WebSocket kapcsolatot és a generálási állapotgépet tartja karban (idle → generating → completed/failed)
- A GenerationFormContext az űrlap mezőinek állapota (cím, szerző, műfajok, beállítások)

A szerver-állapot lekérdezéséhez (generálási előzmények, stílusreferenciák listája, sablonok) a TanStack Query[19] könyvtárat használom, amely automatikus cache-elést, háttérfrissítést és optimista frissítéseket biztosít.

A routing védett útvonalakkal operál: a /generate, /references, /history és /templates oldalak csak autentikált felhasználók számára elérhetők. Nem autentikált felhasználók a /login oldalra irányítódnak.

4.3. Backend architektúra

A backend Flask[4] keretrendszerre épül, Flask-SocketIO kiegészítéssel a valós idejű kommunikációhoz. A belső felépítés a szolgáltatás-réteg mintát követi:

Réteg	Felelősség	Példák
Routes	HTTP végpontok, request validáció	auth.py, generate.py, api_v1.py
Sockets	WebSocket eseménykezelés	handlers.py, tasks.py
Services	Üzleti logika, külső API integráció	pipeline_service.py, llm_service.py
Models	Adatmodellek, szerializáció	Generation, User, StyleReference
Utils	Validáció, segédfüggvények	validation.py, db.py

A szolgáltatások singleton példányok (llm_service, image_service, storage_service), amelyek a Flask alkalmazás kontextusából olvassák ki a konfigurációt az első híváskor (lazy inicializáció).

4.4. Adatmodell

Az adatbázis öt fő táblából áll, PostgreSQL-ben Supabase-en hostolva:

A **Users** tábla felhasználói adatokat tartalmaz: azonosító, Supabase auth ID (google_id), email, név, kreditegyenleg, admin státusz, API token, preferenciák (JSONB).

A **Generations** tábla generálási rekordokat tárol: bemenet (cím, szerző, ötletek, műfaj, leírás), beállítások (képarány, pipeline típus, referencia mód), eredmények (base prompt, text prompt, kép URL-ek), és futási állapot (status, current_step, error_message).

A **StyleReferences** tábla stílusreferencia képeket tartalmaz: eredeti kép útvonala, AI elemzés négy dimenzióban, származtatott variánsok útvonala (clean, text layer), detektált szövegek (JSONB).

A **CoverTemplates** tábla borítósablonokat tárol: felhasználó által definiált szövegdoboz-konfigurációk (title_box, author_box JSONB mezőkben), amelyek pozíciót, betűtípust, méretet és stílust határoznak meg.

Az **Invites** tábla meghívó kódokat tartalmaz: kód, létrehozó, lejárat, felhasználás időpontja. A zárt béta hozzáférés-kezeléséhez használatos.

A táblák közötti kapcsolatok: - generations.user_id → users.id (CASCADE DELETE) - generations.style_reference_id → style_references.id (SET NULL) - generations.cover_template_id → cover_templates.id (SET NULL) - style_references.user_id → users.id (CASCADE DELETE) - cover_templates.user_id → users.id (CASCADE DELETE)

4.5. Deployment architektúra

Az alkalmazás Docker[10] konténerekben fut, két szolgáltatással:

- Az instacover-api egy Python 3.11 alapú konténer Flask + Gunicorn-nal, Playwright/Chromium-mal a sablon-rendereléshez. Port: 5000 (belső), 7710 (külső).
- Az instacover-web egy többlépcsős build: Node.js 20-al fordít, majd Nginx Alpine-nal szolgálja ki a statikus fájlokat. Port: 80 (belső), 7711 (külső).

A konténerek egy közös Docker bridge hálózaton futnak. A külvilág felé Nginx Proxy Manager végzi a reverse proxy feladatot és a TLS terminációt, a domain pedig Cloudflare mögött áll.

A production deployment lépései: 1. docker compose up --build a VPS-en 2. Az Nginx Proxy Manager proxyálja a 7710 és 7711 portokat 3. A Cloudflare biztosítja a DNS-t és az alap DDoS védelmet

4.6. Adatfolyam

Egy tipikus generálási kérés adatfolyama:

1. A felhasználó kitölti az űrlapot → a React app összegyűjti a GenerationInput objektumot
2. A WebSocket start_generation eseményt küld a backend-nek
3. A backend validálja a bemenetet, ellenőrzi a krediteket, létrehoz egy Generation rekordot
4. Háttérfeladat indul, amely a megfelelő pipeline-t futtatja
5. A pipeline LLM hívásokat tesz az OpenRouter-hez → promptot kap vissza
6. A pipeline képgenerálási kéréseket küld a WaveSpeed-nek → polling-gal várja az eredményt
7. A kész képek feltöltődnek a Supabase Storage-ba
8. A backend frissíti az adatbázist és generation_completed eseményt küld a kliensnek

9. A frontend megjeleníti az eredményt signed URL-lel

Minden lépésnél `generation_progress` események tájékoztatják a felhasználót a haladásról.

5. AI generálási pipeline

Az InstaCover magfunkciója a könyvborító automatikus generálása mesterséges intelligencia segítségével. A generálási folyamat nem egyetlen API hívás, hanem több lépés láncolata: egy nagy nyelvi modell (LLM) kreatív promptot generál, egy képgenerálási szolgáltatás ennek alapján elkészíti a vizuális elemet, majd opcionálisan további lépések adják hozzá a tipográfiát. Szakdolgozatomban ezt a többlépéses feldolgozási láncot pipeline-nak nevezem.

A rendszer három különálló pipeline-t implementál, amelyek eltérő felhasználói igényeket szolgálnak ki. Ebben a fejezetben bemutatom mindhárom pipeline működését, a prompttervezés módszertanát, a képgenerálási integráció technikai részleteit, valamint a pipeline-ok közös infrastruktúráját.

5.1. Pipeline architektúra áttekintése

A pipeline-ok tervezésénél az volt a kiindulópontom, hogy a felhasználók igényei jelentősen eltérnek: egyesek csak háttérképet szeretnének (a szöveget saját maguk helyezik el), mások teljes borítót (háttér + tipográfia), megint mások egy meglévő borító stílusát szeretnék reprodukálni. Mindegyik eset eltérő lépésszámot és API-hívás-kombinációt igényel.

A három pipeline:

1. A standard pipeline könyv adatok alapján generál borítót, opcionálisan tipográfiával
2. A stílusreferencia pipeline feltöltött borítókép vizuális stílusát veszi alapul
3. A sablon pipeline HTML/CSS sablonon alapuló tipográfiai réteget ad, szerver oldali rendereléssel

Mindhárom pipeline azonos életciklust követ: a felhasználó bemenete validáláson és kredit-ellenőrzésen megy keresztül, majd a háttérfeladat elindítja a megfelelő pipeline-t. Minden lépés előtt a rendszer ellenőrzi, hogy a felhasználó nem kezdeményezte-e a generálás megszakítását. A haladásról WebSocket eseményeken keresztül kap visszajelzést a kliens.

A pipeline-ok közös segédfüggvényt használnak a szegélydetektáláshoz. A képgenerálási szolgáltatások gyakran adnak egyszínű szegélyt az eredményhez, amit automatikusan eltávolítok:

```
def _check_and_remove_border(image_url, gen_id):
    response = http_requests.get(image_url, timeout=60)
    response.raise_for_status()
    cropped_bytes = detect_and_crop_border(response.content)
```

```

if cropped_bytes is not None:
    upload = storage_service.upload_bytes(cropped_bytes, folder='covers')
    return upload['public_url']
else:
    return image_url

```

A `detect_and_crop_border` függvény a PIL könyvtárral pixel-szinten vizsgálja a kép széléit. Ha a szélső sorok és oszlopok színe megegyezik (egy toleranciaértéken belül), levágja azokat. Ez biztosítja, hogy a végeredmény konzisztens legyen függetlenül attól, hogy a képgeneráló API milyen szegélyt adott hozzá.

A megszakítás-ellenőrzés szintén közös mechanizmus:

```

def _check_cancelled(gen_id):
    result = get_supabase().table('generations').select('status') \
        .eq('id', gen_id).execute()
    if result.data and result.data[0]['status'] != 'generating':
        raise GenerationCancelled(f"Generation #{gen_id} was cancelled")

```

Ez a függvény minden pipeline-lépés előtt lefut. Ha a felhasználó időközben megszakította a generálást (a frontend `cancel_generation` eseményt küldött), a pipeline azonnal leáll, és nem futtatja a további költséges API hívásokat.

5.2. Standard pipeline

A standard pipeline a legegyszerűbb útvonal: a felhasználó megadja könyve adatait (cím, szerző, műfaj, borítóötletek), és a rendszer ezekből generál borítót. Három al módja van, amelyeket a felhasználó a generálási beállításokban választhat ki.

5.2.1. Csak alapkép mód

Ez a legrövidebb útvonal (2 lépés). Az LLM egy szövegmentes háttérképet leíró promptot generál, majd a WaveSpeed API elkészíti a képet.

Az LLM prompt generálása során explicit utasítást kap: ne tartalmazzon szöveget az eredmény. Ezt a generált prompt végéhez fűzött kiegészítéssel is megerősítem:

```

base_prompt = llm_service.generate_base_image_prompt(
    book_data, base_image_only=True, user=user
)
base_prompt += (
    " Do not include any text, words, letters, titles, "
    "or typography anywhere in the image."
)

```

A kettős biztosítás szükséges, mert a képgenerálási modellek hajlamosak szöveget beilleszteni a könyvborítókra akkor is, ha a prompt ezt nem kéri, pusztán azért, mert a tanító adatban gyakran szerepelt szöveg a borítókön.

5.2.2. Egylépéses mód

Az egylépéses mód (szintén 2 lépés a pipeline-ban) egyetlen promptból generál teljes borítót szöveggel együtt. Az LLM ilyenkor a cím és szerzőnév tipográfiai elhelyezését is leírja a promptban. Ez gyorsabb és olcsóbb (1 LLM + 1 kép hívás), de a tipográfiai minőség kevésbé kontrollálható.

5.2.3. Kétlépéses mód

A kétlépéses mód (3 lépés) az alapértelmezett és leggyakrabban használt útvonal:

1. Az LLM szövegmentes háttérkép-promptot generál
2. A WaveSpeed generálja az alapképet
3. Egy második LLM hívás tipográfiai promptot generál, majd a WaveSpeed edit végpontja ráhelyezi a szöveget az alapképre

A harmadik lépésben a rendszer nem egy új képet generál, hanem a WaveSpeed szerkesztő API-ját használja. Ez a végpont egy meglévő képet (referenciaként) és egy szöveges utasítást kap bemenetül:

```
signed_base_url = storage_service.get_signed_url(
    base_storage_path, expires_in=600
)
final_prompt = (
    f"Add the following text to the book cover "
    f"using this base image as reference: {text_prompt}"
)
final_result = image_service.generate_image_with_text(
    signed_base_url, final_prompt, aspect_ratio, user=user
)
```

Ez a megközelítés szétválasztja a vizuális tervezést és a tipográfiát, ami jobb eredményt ad: az alapkép nem torzul a szöveg beszúrásától, a tipográfia pedig a meglévő kompozícióhoz igazodik.

5.3. Sablon pipeline

A sablon pipeline eltérő megközelítést alkalmaz a tipográfiához: a szöveget nem AI-val generáltatja a képre, hanem egy HTML/CSS sablon alapján rendereli szerver oldalon. Ez precíz, pixelpontos kontrollt biztosít a felhasználónak a szöveg elhelyezése, betűtípusa, mérete és stílusa felett.

A pipeline három lépésből áll:

1. LLM generál egy szövegmentes háttérkép-promptot (opcionálisan stílusreferenciával)
2. WaveSpeed elkészíti az alapképet

3. A Playwright böngésző-motor rendereli a végleges borítót

A harmadik lépés a legérdekesebb. A CoverTemplate modell két JSONB mezőt tartalmaz (title_box és author_box), amelyek precízen definiálják a szövegdobozok pozícióját, méretét, betűtípusát és stílusát. Ezekből a rendszer egy teljes HTML oldalt épít:

```
def render_cover_from_template(self, base_image_url, template,
                                book_title, author_name, aspect_ratio='2:3'):
    ratio = ASPECT_RATIOS.get(aspect_ratio, ASPECT_RATIOS['2:3'])
    width, height = ratio['width'], ratio['height']

    html_content = _build_html(
        base_image_url=base_image_url,
        width=width, height=height,
        title_box=title_box, author_box=author_box,
        book_title=book_title, author_name=author_name,
    )

    with sync_playwright() as playwright:
        browser = playwright.chromium.launch(headless=True)
        context = browser.new_context(
            viewport={'width': width, 'height': height},
            device_scale_factor=1,
        )
        page = context.new_page()
        page.set_content(html_content, wait_until='networkidle')
        page.evaluate('() => document.fonts.ready')
        image_bytes = page.locator('#canvas').screenshot(type='png')
        browser.close()
    return image_bytes
```

A generált HTML a háttérképet CSS background-image-ként tölti be, a szövegdobozokat abszolút pozicionálással helyezi el, és a Google Fonts CDN-ről tölt be 14 előre jóváhagyott betűtípust. A `wait_until='networkidle'` biztosítja, hogy a betűtípusok betöltődjenek a screenshot előtt, a `document.fonts.ready` Promise pedig garantálja a renderelés teljességét.

A Playwright használatának előnye, hogy a CSS teljes eszköztára rendelkezésre áll: text-shadow, letter-spacing, text-transform, opacity, és mindez pixelpontosan megjelenik a végleges képen. Hátránya a szerver oldali erőforrásigény: a Docker konténerbe Chromium böngészőt is telepíteni kell.

5.4. Prompt engineering

A generálási minőség döntő mértékben függ az LLM-nek adott promptoktól. Az összes prompt sablon egy központi prompts.json fájlban él, amely a rendszer indításakor egyszer betöltődik és cache-elődik.

5.4.1. Prompt struktúra

Minden LLM hívás két üzenetből áll: egy system prompt határozza meg a szerepet és szabályokat, egy user prompt adja meg a konkrét feladatot. A válasz formátumát JSON Schema-val kényszerítem ki:

```
payload = {
    'model': model,
    'messages': messages,
    'temperature': 0.7,
    'max_tokens': 2000,
}
if schema:
    payload['response_format'] = {
        'type': 'json_schema',
        'json_schema': schema,
    }
```

A JSON Schema használata garantálja, hogy az LLM mindig géppel feldolgozható formátumban válaszoljon. Ha a modell mégis hibás JSON-t ad vissza (ritka, de előfordul), egy többlepcsős javítási mechanizmus próbálja helyreállítani: először standard parse, majd kódblokk-kivonás, végül a json_repair könyvtár.

5.4.2. Modellválasztás

A rendszer két LLM modellt használ. A Grok 4.1 Fast az alapértelmezett modell promptgeneráláshoz, gyors válaszidővel és jó teljesítménnyel a kreatív szövegalkotásban. A Gemini 3 Flash kizárólag képelemzési feladatokhoz használatos (stílusanalízis, szövegdetektálás, réteg-verifikáció), mert multimodális képességei kifinomultabbak.

Mindkét modell az OpenRouter[8] API-n keresztül érhető el, ami egy egységes felületet biztosít különböző LLM szolgáltatók modelljeihez. A választásom azért esett az OpenRouter-re, mert egyetlen API kulccsal több tucat modell elérhető, és modellváltás kódmódosítás nélkül lehetséges.

5.4.3. Könyvadatok injektálása

A felhasználó által megadott könyvadatokat strukturáltan építem be a promptba:

```
def _build_book_details_content(book_data, include_title=True):
    parts = []
    if include_title and book_data.get('book_title'):
        parts.append(f"Title: {book_data.get('book_title')}")
    if book_data.get('cover_ideas'):
        parts.append(
            f"Cover Ideas (author's vision): {book_data.get('cover_ideas')}"
        )
    if book_data.get('description'):
        parts.append(f"Description: {book_data.get('description')}")
    if book_data.get('genres'):
```

```
parts.append(f"Genre(s): {'', ' '.join(book_data.get('genres', []))}")
return "\n".join(parts)
```

A “Cover Ideas” mező különösen fontos: itt a felhasználó saját elképzelését adja meg (pl. “sötét erdő, holdvilág, egy magányos alak”), amely közvetlenül irányítja a vizuális generálást. Ez a mező opcionális, és ha nincs megadva, az LLM a cím, műfaj és leírás alapján önállóan tervez.

5.5. Képgenerálási integráció

A képgenerálást a WaveSpeed AI[9] szolgáltatás végzi, amely a ByteDance Seedream V4.5 modelljét üzemelteti. Ez egy korszerű text-to-image modell, amely különösen jól kezeli a könyvborítókra jellemző igényeket: magas felbontás, szöveges elemek renderelése, konzisztens stílusú illusztrációk.

5.5.1. Aszinkron job-kezelés

A WaveSpeed API aszinkron működésű: a kérés beküldésekor azonnal visszaad egy job azonosítót, a kép generálása pedig háttérben történik. A rendszernek polling mechanizmussal kell lekérdeznie az eredményt:

```
def _submit(self, url, payload, user=None):
    r = requests.post(
        url, headers=self._headers(), json=payload, timeout=30
    )
    job_id = r.json()["data"]["id"]
    return job_id

def _poll(self, job_id, interval=1):
    url = f"{self.base_url}/predictions/{job_id}/result"
    poll_count = 0
    while True:
        r = requests.get(url, headers=self._headers(), timeout=30)
        data = r.json()["data"]
        poll_count += 1

        if data["status"] == "completed":
            return data["outputs"][0]
        if data["status"] == "failed":
            raise RuntimeError(f"Job failed: {data.get('error')}")
        if poll_count >= 120:
            raise RuntimeError("Job timed out")
        time.sleep(interval)
```

A polling másodpercenként kérdez, és legfeljebb 120 próbálkozás után timeout hibát dob. Ez biztosítja, hogy egy elakadt job ne blokkolja végtelenül a rendszert.

5.5.2. Két API végpont

A WaveSpeed két végpontot biztosít. A `/bytedance/seedream-v4.5` végpont text-to-image generálást végez: bemenete egy szöveges prompt és a kívánt méret. A `/bytedance/seedream-`

v4.5/edit végpont képszerkesztésre szolgál: bemenete egy vagy több referenciakép, egy szöveges utasítás, és a kívánt méret. Ez utóbbi a tipográfia hozzáadásához és a stílusreferencia-alapú generáláshoz használatos.

A méret meghatározásánál a felhasználó által választott képarányt (pl. 2:3, 3:2, 1:1) pixel-értékekre képezem le egy előre definiált táblázat alapján.

5.6. Kredit-integráció a pipeline-ban

A pipeline-ok lépésenként vannak le krediteket. Nem előre számolom ki és vonom le a teljes költséget, hanem minden egyes API híváskor külön-külön történik a levonás. Ennek oka, hogy ha egy közbelső lépés elbukik, a felhasználó csak az addig felhasznált lépések kreditjét veszíti el. Ha a kredit nem elegendő, az `InsufficientCreditsError` kivétel azonnal leállítja a pipeline-t, és a felhasználó hibaüzenetet kap a WebSocket-en. A kreditrendszer részletes működését, az atomikus levonás implementációját és a dinamikus költségbecslést a 8. fejezet tárgyalja.

6. Stílusreferencia rendszer

Az InstaCover egyik legértékesebb funkciója a stílusreferencia rendszer: a felhasználó feltölthet meglévő könyvborítókat, amelyek vizuális stílusát a rendszer AI segítségével elemzi és reprodukálja új generálásoknál. Ez lehetővé teszi, hogy egy sorozat borítói vizuálisan konzisztensek maradjanak, vagy hogy a felhasználó egy tetszőleges borító “hangulatát” újrahasznosítsa saját könyvéhez.

6.1. Stíluselemzés feltöltéskor

Amikor a felhasználó feltölt egy stílusreferencia képet, a rendszer azonnal AI-alapú elemzést futtat rajta. A Gemini 3 Flash multimodális modell négy dimenzió mentén dekompozícionálja a borítót:

- Az érzelmi hangulat és atmoszféra (feeling), például “sötét, misztikus, feszültségkeltő”
- A kompozíciós szerkezet és térelosztás (layout), például “központi alak, felső harmadban cím”
- A médium, művészeti technika és színpaletta (illustration rules), például “digitális festmény, hideg tónusok, kontrasztos megvilágítás”
- A betűtípus-választás, hierarchia és kezelés (typography), például “nagy serif cím, kis sans-serif szerzőnév”

Az elemzés JSON Schema-val kényszerített struktúrában érkezik:

```
def analyze_style_reference(self, image_url, user=None):
    system_prompt = get_prompt('style_analysis', 'system')
    user_text = get_prompt('style_analysis', 'user_template')

    messages = [
        {'role': 'system', 'content': system_prompt},
        {
            'role': 'user',
            'content': [
                {'type': 'text', 'text': user_text},
                {'type': 'image_url', 'image_url': {'url': image_url}},
            ],
        },
    ]
    result = self._make_request(
        messages, schema=get_prompt_schema('style_analysis'),
        model=STYLE_ANALYSIS_MODEL, user=user,
    )
    return result
```

A multimodális hívásban a kép URL-ként kerül az üzenetbe, így a modell közvetlenül “látja” a borítót és szöveges analízist ad vissza. Az eredmény a StyleReference modellben tárolódik, ahol minden dimenzió külön mezőt kap.

6.2. Szövegdetektálás és szelekció

A stílselemzés mellett a rendszer szövegdetektálást is végez a feltöltött képen. Ez azonosítja a borítón található szöveges elemeket (cím, szerzőnév, alcím, kiadó stb.) és metaadatokkal látja el őket:

```
{
  "detected_texts": [
    {
      "id": 1,
      "text": "THE MIDNIGHT GARDEN",
      "text_type": "title",
      "position": "top-center",
      "style_description": "large serif white text with shadow"
    },
    {
      "id": 2,
      "text": "Sarah Mitchell",
      "text_type": "author_name",
      "position": "bottom-center",
      "style_description": "small sans-serif gold text"
    }
  ]
}
```

A felhasználó a frontend-en kiválaszthatja, mely szövegelemeket szeretné megtartani a tipográfiai stílus reprodukálásánál. Ez a `selected_text_ids` mezőben tárolódik. Ha a felhasználó például csak a cím stílusát szeretné átvenni (és az alcímet vagy kiadót nem), ezt itt jelölheti meg.

6.3. Referencia variánsok

A stílusreferencia pipeline működéséhez a rendszernek két származtatott képre van szüksége az eredeti borítóból:

1. A clean variáns az eredeti kép szöveg nélkül (csak háttérgrafika)
2. A text variáns a szöveg kiemelve fehér háttérre (csak tipográfia)

Ezek előállítására költséges (képgenerálási API hívások), ezért a rendszer lusta kiértékeléssel és cache-eléssel dolgozik:

```
def ensure_reference_variant(style_ref, variant_type, user_id, user=None):
    if variant_type == 'clean':
        if style_ref.clean_image_path:
            return storage_service.get_signed_url(
                style_ref.clean_image_path, expires_in=600
            )

        signed_original = storage_service.get_signed_url(
            style_ref.image_path, expires_in=600
        )
        result = image_service.generate_clean_background(
            signed_original, user=user
        )
        upload = storage_service.upload_from_url(
```

```

        result['image_url'], folder='references'
    )
    get_supabase().table('style_references').update({
        'clean_image_path': upload['path']
    }).eq('id', style_ref.id).eq('user_id', user_id).execute()
    return storage_service.get_signed_url(upload['path'], expires_in=600)

```

Az első használatkor a variáns legenerálódik és az adatbázisban cache-elődik. Minden további generálásnál a cache-elt verzió töltődik be, így nem kell újra fizetni a variáns előállításáért. Ez a megoldás különösen fontos, mert egy stílusreferenciát a felhasználó tipikusan többször is felhasznál, minden új borítógenerálásnál.

6.3.1. Text layer verifikáció

A text variáns előállítása után a rendszer egy extra minőségellenőrzési lépést hajt végre. Az LLM megvizsgálja a generált text layert, és ellenőrzi, hogy valóban csak szöveget tartalmaz-e fehér háttéren:

```

verification = llm_service.verify_text_layer(variant_url, user=user)
if not verification.get('is_clean', True) and verification.get('artifacts'):
    artifacts = verification['artifacts']
    cleanup_result = image_service.cleanup_text_layer(
        variant_url, artifacts_desc, text_details=text_details, user=user
    )
    variant_url = cleanup_result['image_url']

```

Ha a verifikáció artifaktumokat talál (például háttérelemek maradványait vagy torz szöveget), automatikusan elindít egy tisztítási lépést, amely eltávolítja ezeket. Ez a két fázisú ellenőrzés biztosítja, hogy a text layer valóban használható legyen a későbbi blending műveleteknél.

6.4. Referencia módok

A felhasználó három módban használhatja a stílusreferenciát:

Mód	Felhasznált elemzés	Hatás
both	feeling + layout + illustration_rules + typography	Teljes stílus átvétel
background	feeling + layout + illustration_rules	Csak vizuális elemek, új tipográfia
text	typography	Csak tipográfiai stílus, új vizuális

A mód választás közvetlenül befolyásolja, hogy az elemzés mely részeit injektáljuk az LLM promptba:

```

def get_style_analysis(self, mode: str = 'both') -> dict:
    if mode == 'text':
        return {'typography': self.typography or ''}
    elif mode == 'background':
        return {
            'feeling': self.feeling or '',
            'layout': self.layout or '',
            'illustration_rules': self.illustration_rules or '',

```

```

    }
    return {
        'feeling': self.feeling or '',
        'layout': self.layout or '',
        'illustration_rules': self.illustration_rules or '',
        'typography': self.typography or ''
    }

```

6.5. Text blending módok

A kétlépéses stílusreferencia pipeline utolsó fázisában, amikor a szöveget kell ráhelyezni az alapképre, három blending módszer közül választhat a felhasználó:

6.5.1. AI Blend

Az alapértelmezett és legösszetettebb mód. Két WaveSpeed hívást használ:

1. Az első összefésüli az alapképet és a text layert (a referencia tipográfiájával)
2. Egy LLM szövegcsere promptot generál, majd egy második WaveSpeed hívás lecseréli a referencia szövegeit a felhasználó könyvének adataira

Ez a mód adja a legjobb vizuális eredményt, mert az AI természetesen integrálja a szöveget a háttérrel, de a legdrágább is (extra LLM + 2 kép hívás).

6.5.2. Direct Overlay

Programmatikus megoldás PIL könyvtárral: a text layer fehér pixeleit átlátszóvá teszi, majd alpha-kompozícióval a base image-re helyezi. Utána egyetlen WaveSpeed cleanup hívás javítja az esetleges artifaktumokat:

```

def blend_images_programmatic(base_image_url, text_layer_url,
                              white_threshold=240):
    base_response = http_requests.get(base_image_url, timeout=60)
    text_response = http_requests.get(text_layer_url, timeout=60)
    base_img = Image.open(io.BytesIO(base_response.content)).convert('RGBA')
    text_img = Image.open(io.BytesIO(text_response.content)).convert('RGBA')
    width, height = text_img.size

    text_data = text_img.load()
    for y in range(height):
        for x in range(width):
            r, g, b, a = text_data[x, y]
            if r >= white_threshold and g >= white_threshold \
                and b >= white_threshold:
                text_data[x, y] = (r, g, b, 0)
            else:
                brightness = (r + g + b) / 3
                opacity = int(255 * (1 - brightness / 255))
                text_data[x, y] = (r, g, b, min(255, opacity + 50))

    result = Image.alpha_composite(base_img, text_img)
    return result

```

A fehér háttér eltávolítása egy küszöbértékkel történik: minden pixel, amelynek RGB értékei meghaladják a 240-et, átlátszóvá válik. A sötétebb pixelek opacitása arányosan növekszik, így a szöveg természetesen jelenik meg az alapképen.

6.5.3. Separate Reference

A legegyszerűbb AI-alapú mód: egyetlen WaveSpeed edit hívás kap két referenciaképet (az alapképet és a text layert) és egy utasítást, hogy egyesítse őket. Ez gyorsabb és olcsóbb, mint az AI Blend, de kevesebb kontrollal rendelkezik a szövegcsere felett.

7. Valós idejű kommunikáció

A könyvborító generálása 15-60 másodpercig tart a pipeline típusától és a lépések számától függően. Ez az időtartam túl hosszú ahhoz, hogy a felhasználó visszajelzés nélkül várakozzon, szüksége van arra, hogy lépésről lépésre lássa a haladást. A hagyományos HTTP kérés-válasz modell erre nem alkalmas: a hosszú pollozás pazarló, a szerver oldali események (SSE) pedig egyirányúak. A választásom a WebSocket protokollra[16] esett, konkrétan a Socket.IO[6] könyvtárra, amely kétirányú, valós idejű kommunikációt biztosít.

7.1. Kommunikációs architektúra

A rendszer a Flask-SocketIO kiszolgálót és a Socket.IO JavaScript klienst használja. A kommunikáció szoba-alapú (room-based): minden felhasználó egy `user_{id}` nevű szobába kerül csatlakozáskor, és az összes őt érintő esemény ebbe a szobába érkezik.

A backend oldalon a Flask-SocketIO natívan integrálódik a Flask alkalmazásba:

```
@socketio.on('connect')
def handle_connect(auth=None):
    token = auth.get('token') if auth and isinstance(auth, dict) else None
    if not token:
        disconnect()
        return False

    supabase_user = get_user_from_token(token)
    if not supabase_user:
        disconnect()
        return False

    user = User.from_row(db_result.data[0])
    connected_users[request.sid] = user
    join_room(_room_for(user.id))
```

A csatlakozás során három dolog történik: JWT token validáció, a felhasználó beazonosítása az adatbázisból, és a szobához való csatlakozás. Ha bármelyik lépés sikertelen, a kapcsolat azonnal bontásra kerül.

7.2. Eseménymodell

A rendszer hét fő eseménytípust definiál:

Esemény	Írány	Leírás
start_generation	kliens → szerver	Generálás indítása
generation_started	szerver → kliens	Visszaigazolás, generálás elindult
generation_progress	szerver → kliens	Lépésenkénti haladás
generation_completed	szerver → kliens	Végleges eredmény
generation_failed	szerver → kliens	Hiba történt
cancel_generation	kliens → szerver	Generálás megszakítása

Esemény	Irány	Leírás
active_generation	szerver → kliens	Aktív generálás visszaállítása reconnect után

A `generation_progress` esemény tartalmazza az aktuális lépés sorszámát, az összes lépés számát, és egy szöveges üzenetet:

```
def on_progress(step, total, message):
    get_supabase().table('generations').update({
        'current_step': step,
        'total_steps': total,
        'step_message': message,
    }).eq('id', gen_id).execute()
    socketio.emit('generation_progress', {
        'generation_id': gen_id,
        'step': step,
        'total_steps': total,
        'message': message,
    }, room=room)
```

A haladási információ az adatbázisba is íródik, nem csak a WebSocket-re kerül. Ennek oka, hogy ha a kliens kapcsolata megszakad és újracsatlakozik, a szerver az adatbázisból képes visszaállítani az aktuális állapotot.

7.3. Háttérfeladat-kezelés

A generálási pipeline nem a WebSocket eseménykezelő szálában fut, mert az túl sokáig blokkolná a szerver válaszképességét. Ehelyett a Flask-SocketIO `start_background_task` módszerével indítok háttérfeladatot:

```
socketio.start_background_task(
    _run_generation_task,
    current_app.get_current_object(),
    generation, user,
    generation.style_reference_id,
    generation.use_style_image,
    generation.aspect_ratio,
    generation.base_image_only,
    generation.reference_mode,
    text_blending_mode,
    generation.cover_template_id,
)
```

A háttérfeladat megkapja az alkalmazás kontextust (szükséges a konfiguráció és adatbázis eléréséhez), a generálás és felhasználó objektumokat, valamint az összes konfigurációs paramétert. A feladat futása alatt a szerver szabad marad más kérések kiszolgálására.

A duplikált futás megelőzésére egy `_running_tasks` halmaz tartja számon az aktív generálás-azonosítókat:

```
_running_tasks = set()

def _run_generation_task(app, generation, user, ...):
    with app.app_context():
        gen_id = generation.id
```

```

    if gen_id in _running_tasks:
        return
    _running_tasks.add(gen_id)
    try:
        ...
    finally:
        _running_tasks.discard(gen_id)

```

7.4. Hibakezelés és állapot-konzisztencia

A háttérfeladat három különböző hibatípust kezel eltérő módon:

```

except Exception as e:
    if isinstance(e, GenerationCancelled):
        return

    if isinstance(e, InsufficientCreditsError):
        error_message = (
            f"Insufficient credits: need {e.required}, "
            f"have {e.available}"
        )
    else:
        error_message = 'Generation failed. Please try again.'

    get_supabase().table('generations').update({
        'status': 'failed',
        'error_message': error_message,
    }).eq('id', gen_id).execute()

    socketio.emit('generation_failed', {
        'generation_id': gen_id,
        'error': error_message,
    }, room=room)

```

A `GenerationCancelled` kivétel esetén a felhasználó megszakította a folyamatot, így a feladat csendben leáll, mert a megszakítás kezelés már frissítette az adatbázist. Az `InsufficientCreditsError` azt jelzi, hogy elfogytak a kreditek közben, és specifikus hibaüzenetet küld a felhasználónak. Minden egyéb kivétel (API timeout, hálózati hiba stb.) esetén a felhasználó generikus hibaüzenetet kap, míg a szerveren részletes log keletkezik.

Minden esetben az adatbázis konzisztens állapotba kerül (`status='failed'`), és a kliens értesül a hibáról.

7.5. Reconnect és állapot-visszaállítás

Ha a felhasználó böngészője újratöltődik vagy a hálózati kapcsolat átmenetileg megszakad, a `Socket.IO` automatikusan megpróbálja újra felépíteni a kapcsolatot (legfeljebb 10 próbálkozás, exponenciális backoff). Újracsatlakozáskor a szerver ellenőrzi, hogy van-e aktív generálás:

```

active = _check_active_generation(user.id)
if active:
    emit('active_generation', {
        'generation_id': active.id,

```



```

    'book_title': active.book_title,
    'step': active.current_step or 0,
    'total_steps': active.total_steps or 0,
    'step_message': active.step_message or 'Resuming generation...',
  })

```

Ez biztosítja, hogy a felhasználó soha ne veszítse el a generálási folyamat állapotát: akár szándékos oldalújrátöltés, akár hálózati probléma esetén a progress panel azonnal visszaáll az aktuális lépéshez.

Emellett, ha a generálás az elmúlt 2 percben fejeződött be, a szerver a `generation_completed` eseményt is elküldi, így a felhasználó akkor sem veszít eredményt, ha éppen a befejezés pillanatában szakadt meg a kapcsolata.

7.6. Stale detection

Ha egy generálás valamilyen okból elakad (szerver újraindulás, kezeletlen kivétel a háttérfeladatban), a rendszer nem hagyja végtelen “generating” állapotban. Minden alkalommal, amikor aktív generálást keres, ellenőrzi annak korát:

```

STALE_TIMEOUT_MINUTES = 5

def _is_stale(generation):
    created = generation.created_at
    age_minutes = (datetime.now(timezone.utc) - created).total_seconds() / 60
    return age_minutes > STALE_TIMEOUT_MINUTES

```

Az 5 percnél régebbi, még mindig “generating” státuszú rekordok automatikusan “failed” állapotra váltanak. Ez megakadályozza, hogy egy felhasználó végleg blokkolva maradjon, és lehetővé teszi, hogy új generálást indítson.

7.7. Frontend állapotgép

A kliens oldalon a `GenerationContext` React kontextus kezeli a WebSocket kommunikációt és a generálási állapotot. Az állapotgép négy állapotot definiál: `idle` (nincs aktív generálás, a felhasználó kitöltheti az űrlapot), `generating` (generálás folyamatban, a progress panel látható), `completed` (generálás kész, az eredmény megtekinthető) és `failed` (hiba történt, hibaüzenet látható).

Az átmenetek kizárólag szerver-eseményeken alapulnak:

```

socket.on('generation_progress', (data) => {
  setStep(data.step);
  setTotalSteps(data.total_steps);
  setStepMessage(data.message);
});

socket.on('generation_completed', (data) => {
  setStatus('completed');

```

```
    setResult(data.generation);
  });

socket.on('generation_failed', (data) => {
  setStatus('failed');
  setError(data.error);
});
```

A generálás indítása is WebSocket-en történik (nem REST hívás):

```
const startGeneration = useCallback((data: GenerationInput) => {
  const socket = getSocket();
  if (!socket?.connected) {
    setError('Not connected to server. Please refresh the page.');
```

return;
 }
 socket.emit('start_generation', data);
}, []);

Ez a döntés leegyszerűsíti az architektúrát: a teljes generálási folyamat egyetlen csatornán zajlik, nincs szükség HTTP és WebSocket közötti koordinációra.

7.8. Rate limiting

A WebSocket eseményekre is alkalmazok rate limitinget, hogy megakadályozzam a visszaélést. Egy felhasználó legfeljebb 10 eseményt küldhet 60 másodperc alatt, sliding window algoritmussal. Ha a limit túllépésre kerül, a felhasználó generation_error eseményt kap és a kérése elutasítódik. A rate limiting implementáció részleteit és a többi védelmi réteghez való kapcsolódását a 8. fejezet ismerteti.

8. Biztonság és kreditrendszer

Egy publikusan elérhető webalkalmazásnál, amely költséges külső API-kat hív (LLM és képgenerálás), a biztonság és a költségkontroll kritikus. Ebben a fejezetben bemutatom az autentikációs rendszert, a több szintű jogosultság-kezelést, a bemeneti validációt, a prompt injection elleni védelmet és az atomikus kreditrendszert.

8.1. Autentikáció

Az autentikációt a Supabase Auth[5] szolgáltatás végzi, amely JWT token alapú session kezelést biztosít. Két belépési módot támogatok: email/jelszó és Google OAuth.

A backend kétféle token-validációt implementál:

JWT token (felhasználói hozzáférés): Minden REST kérés és WebSocket csatlakozás egy Supabase által kibocsátott JWT-t tartalmaz. A backend a Supabase SDK-val validálja:

```
def get_user_from_token(token):
    response = current_app.supabase.auth.get_user(token)
    return response.user
```

API token (programozói hozzáférés): Admin felhasználók számára elérhető, ic_ prefixű, 64 hexadecimális karakterből álló token. Ez lehetővé teszi a generálási API programozói elérését (pl. automatizáláshoz):

```
API_TOKEN_PREFIX = 'ic_'
API_TOKEN_BYTES = 32

def generate_api_token():
    return API_TOKEN_PREFIX + secrets.token_hex(API_TOKEN_BYTES)
```

8.2. Meghívó rendszer

Az alkalmazás zárt bétában működik: regisztrálni kizárólag érvényes meghívóköddel lehet. A meghívóködök kriptográfiailag biztonságos, URL-kompatibilis tokenek (16 byte, secrets.token_urlsafe), 7 napos lejárattal.

A meghívó felhasználása atomikus adatbázis-művelet:

```
CREATE OR REPLACE FUNCTION public.consume_invite(
    p_code TEXT, p_google_id TEXT
) RETURNS BOOLEAN
LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE invite_id BIGINT;
BEGIN
    UPDATE public.invites
    SET used_at = NOW(), used_by_google_id = p_google_id
    WHERE code = p_code
    AND used_at IS NULL
```

```
        AND expires_at > NOW()
    RETURNING id INTO invite_id;

    RETURN invite_id IS NOT NULL;
END; $$;
```

Az UPDATE ... WHERE used_at IS NULL biztosítja, hogy egy kódot csak egyszer lehessen felhasználni, még párhuzamos kérések esetén is.

8.3. Row-Level Security

A Supabase PostgreSQL-ben Row-Level Security (RLS)[15] policy-kat alkalmazok minden táblán. Ez az adatbázis szintű védelmi réteg garantálja, hogy egy felhasználó kizárólag saját adatait érheti el:

```
ALTER TABLE public.generations ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can only access own generations"
    ON public.generations FOR ALL
    USING (user_id = (
        SELECT id FROM public.users
        WHERE google_id = auth.uid()::text
    ))
    WITH CHECK (user_id = (
        SELECT id FROM public.users
        WHERE google_id = auth.uid()::text
    ));
```

Ez a minta minden táblán azonos: a users tábla google_id = auth.uid()::text feltétellel, a többi tábla a user_id mezőn keresztül, alkérdéssel visszavezetve az autentikált felhasználóra.

Az RLS kiegészíti, nem helyettesíti az alkalmazás szintű szűrést. A backend kódban is minden lekérdezés tartalmazza a .eq('user_id', user.id) feltételt. Ez a “defense in depth” elv: ha az alkalmazás szintű szűrés hibás lenne, az adatbázis szintű védelem még megáll.

8.4. Input validáció és prompt injection

Mivel a felhasználó szöveges bemenetei LLM promptokba kerülnek, a prompt injection[17] reális támadási vektor. Egy rosszindulatú felhasználó megpróbálhatja az LLM viselkedését módosítani a bemeneti mezőkön keresztül.

A védekezés két rétegből áll. Először a bemenet sanitizálása: vezérlőkértékek eltávolítása, hossz-limitálás:

```
MAX_TITLE_LENGTH = 200
MAX_LONG_TEXT_LENGTH = 2000

def sanitize_text(value, max_length=MAX_SHORT_TEXT_LENGTH):
    if not isinstance(value, str):
        return None
    value = value.strip()
```

```
value = re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]', '', value)
return value[:max_length]
```

Majd regex-alapú mintaillesztés a gyakori injection technikákra:

```
INJECTION_PATTERNS = re.compile(
    r'(?::ignore\s+(?:all\s+)?(?:previous|above|prior)\s+instructions?'
    r'|you\s+are\s+now\s+(?:a|an)\s+'
    r'|system\s*:\s*'
    r'|<\s*(?:script|system|admin)'
    r'|\\bdAN\\b.*\\bjailbreak\\b'
    r'|override\s+(?:all\s+)?(?:safety|content|policy))',
    re.IGNORECASE,
)
```

Ha a mintaillesztés pozitív eredményt ad, a generálás elutasításra kerül. Ez nem tökéletes védelem (a prompt injection nem oldható meg regex-szel teljes bizonyossággal), de az ismert, gyakori támadási mintákat kiszűri.

A harmadik védelmi vonal az, hogy a felhasználó szövege soha nem kerül közvetlenül a system promptba, mindig a user prompton belül, strukturált formátumban jelenik meg.

8.5. Rate limiting

Több szintű rate limitinget alkalmazok a visszaélések megelőzésére. A REST API végpontokon Flask-Limiter biztosítja a korlátokat: 30 kérés/perc az általános végpontokon, 10/perc a kredit-kiosztáson, 5/perc az API token generáláson. A WebSocket eseményekre egyéni implementáció él: 10 esemény/60 másodperc felhasználónként, sliding window algoritmussal. Emellett a kreditrendszer természetes korlátként is funkcionál, hiszen ha elfogynak a kreditek, nem indítható több generálás.

8.6. Kreditrendszer

A kreditrendszer kettős célt szolgál: költségkontrollt biztosít (minden generálás valós API költséggel jár), és fair használatot ösztönöz a felhasználók között.

8.6.1. Költségstruktúra

Művelet	Költség	Indoklás
LLM hívás	1 kredit	Alacsony API költség (~\$0.001)
Képgenerálás	6 kredit	Magasabb API költség (~\$0.03)
Kezdő egyenleg	200 kredit	~15-20 teljes generálásra elegendő

8.6.2. Atomikus műveletek

A kredit-levonás race condition-érzékeny: ha két párhuzamos kérés egyszerre próbál levonni, az egyenleg negatívba mehet. Ezt PostgreSQL SELECT ... FOR UPDATE sorsintű zárolással oldom meg:

```
CREATE OR REPLACE FUNCTION public.deduct_credits(  
    p_user_id BIGINT, p_amount INT  
) RETURNS INT  
LANGUAGE plpgsql SECURITY DEFINER AS $$  
DECLARE  
    current_credits INT;  
    new_credits INT;  
BEGIN  
    SELECT credits INTO current_credits  
    FROM public.users WHERE id = p_user_id  
    FOR UPDATE;  
  
    IF current_credits IS NULL OR current_credits < p_amount THEN  
        RETURN NULL;  
    END IF;  
  
    UPDATE public.users  
    SET credits = credits - p_amount, updated_at = NOW()  
    WHERE id = p_user_id AND credits >= p_amount  
    RETURNING credits INTO new_credits;  
  
    RETURN new_credits;  
END; $$;
```

A FOR UPDATE zárolja a sort az olvasástól a tranzakció végéig. Az UPDATE feltételében ismételt credits >= p_amount ellenőrzés biztosítja, hogy még ha valami hiba miatt a zárolás nem lenne hatékony, az egyenleg akkor sem megy negatívba.

8.6.3. Lépésenkénti levonás

A kreditek nem egyben, hanem lépésenként vonódnak le a pipeline futása közben. Ha egy közbülső lépés elbukik, a felhasználó csak az addig ténylegesen elhasznált krediteket veszíti el. Ez méltányosabb, mint az előre levonás + visszatérítés modell.

8.6.4. Dinamikus költségbecslés

A frontend a generálás indítása előtt költségbecslést kér a backend-től, amely a kiválasztott beállítások (pipeline típus, referencia mód, blending mód, cache-elt variánsok megléte) alapján számítja a várható költséget:

```
def calculate_generation_cost(  
    use_style_image, base_image_only, reference_mode,  
    text_blending_mode, style_ref_has_clean, style_ref_has_text,  
    two_step_generation, use_template,  
) -> dict:  
    llm_calls = 0  
    image_calls = 0
```

```
if use_template:
    llm_calls, image_calls = 1, 1
elif base_image_only:
    llm_calls, image_calls = 1, 1
elif use_style_image:
    llm_calls, image_calls = 2, 2
    if reference_mode in ('both', 'background') \
        and not style_ref_has_clean:
        image_calls += 1
    ...

total = llm_calls * LLM_CALL_COST + image_calls * IMAGE_CALL_COST
return {'llm_calls': llm_calls, 'image_calls': image_calls, 'total': total}
```

A `style_ref_has_clean` és `style_ref_has_text` paraméterek figyelembe veszik, hogy a variánsok már cache-elve vannak-e, és ha igen, azokért nem kell fizetni újra.

9. Tesztelés és CI/CD

A projekt minőségbiztosítása automatizált tesztekre és folyamatos integrációra épül. Ebben a fejezetben bemutatom a tesztelési stratégiát, a teszt infrastruktúrát és a CI/CD pipeline konfigurációját.

9.1. Tesztelési stratégia

A tesztelés a tesztpiramis elvét követi: a hangsúly a backend unit és integrációs teszteken van, amelyeket automatikus TypeScript type-checking és manuális béta tesztelés egészít ki.

Szint	Eszköz	Lefedett terület
Unit + integráció	pytest	Backend szolgáltatások, route-ok, modellek
Típusellenőrzés	TypeScript (tsc --noEmit)	Frontend típusbiztonság
Linting	flake8	Backend kódminőség
Manuális	Béta tesztelők	E2E felhasználói folyamatok

A backend teszt suite 204 tesztet tartalmaz 14 fájlban, amelyek lefedik az autentikációt, a generálási route-okat, az API v1 végpontokat, a pipeline szolgáltatásokat, a WebSocket kezelőket és a modellek szerializációját.

9.2. Mock infrastruktúra

A tesztek nem hívnak külső szolgáltatásokat (Supabase, OpenRouter, WaveSpeed). Ehelyett egy egyéni in-memory Supabase mock-ot implementáltam a conftest.py-ban:

```
class MockQueryBuilder:
    def select(self, columns='*'): ...
    def insert(self, data): ...
    def update(self, data): ...
    def delete(self): ...
    def eq(self, column, value): ...
    def order(self, column, desc=False): ...
    def limit(self, count): ...
    def execute(self): ...
```

A MockQueryBuilder egy Python dictionary-ben tárolja az adatokat és szimulálja a Supabase táblaműveleteket szűréssel, rendezéssel és paginációval. A MockRpcBuilder az atomikus RPC hívásokat implementálja (kredit-levonás, visszatérítés, meghívó felhasználás).

Ez a megközelítés előnyösebb, mint a valódi adatbázis használata tesztekben: gyorsabb, nem igényel hálózati kapcsolatot, és minden teszt garantáltan tiszta állapotból indul.

9.3. Teszt típusok

A pozitív tesztek sikeres folyamatokat ellenőriznek: generálás indítása érvényes paraméterekkel, helyes autentikáció, kredit-levonás és visszatérítés.

A negatív tesztek a hibakezelést vizsgálják: hiányzó vagy érvénytelen token (401), jogosulatlan hozzáférés (403), hibás bemenet (400), elégtelen kredit (402), nem létező erőforrás (404).

A biztonsági tesztek más felhasználó adatainak elérési kísérletét (user isolation), admin-only végpontok nem-admin felhasználóval való hívását és lejárt tokenek kezelését fedik le.

Az izolációs tesztek biztosítják, hogy minden teszt független: nincs megosztott állapot, a fixture-ök minden futáskor újrainicializálják a mock adattárat.

9.4. CI/CD pipeline

A GitHub Actions[11] CI pipeline minden push és pull request esetén három párhuzamos jobot futtat:

1. A backend-test job Python 3.11 környezetben pytest futtatást végez coverage méréssel, az eredmény feltöltődik Codecov-ra
2. A frontend-build job Node.js 20 környezetben TypeScript type check-et (tsc --noEmit) és production buildet (npm run build) futtat
3. A lint job flake8 futtatást végez a backend kódon (figyelmeztetésként, nem blokkoló)

A deployment manuális: docker compose up --build a VPS-en. Automata deployment-et nem implementáltam, mert a zárt béta jellegű alkalmazásnál a manuális ellenőrzés előnyösebb.

10. Mesterséges intelligencia használata a fejlesztés során

Az InstaCover fejlesztése során a mesterséges intelligenciát nem alkalmi segédeszközként, hanem a napi fejlesztési munkafolyamat szerves részeként használtam. A célom nem az volt, hogy a rendszer helyettem hozzon mérnöki döntéseket, hanem az, hogy felgyorsítsa a megvalósítást azokban a feladatokban, ahol a probléma már kellően pontosan definiálható volt. A fejlesztési folyamatban az MI leginkább tervezési partnerként, kódolóállító eszközként és gyors iterációt támogató asszisztensként működött.

10.1. Felhasznált eszközök

A fejlesztés során három modellt használtam rendszeresen: a Claude Sonnet 4.6-ot, a Claude Opus 4.6-ot és a GPT-5.5-öt. Ezeket minden esetben az OpenCode CLI-n keresztül értem el, vagyis a fejlesztési munkafolyamaton belül, közvetlenül a kódbázis kontextusában dolgoztam velük. A különböző modelleket nem élesen elkülönített szerepekben használtam, hanem attól függően váltottam köztük, hogy egy adott feladtnál melyik adott jobb tervet, pontosabb kódot vagy használhatóbb javaslatot.

A fejlesztéshez kapcsolódó kutatási és tájékoztatósi feladatoknál a Perplexityt is használtam. Ezt nem közvetlen kódgenerálásra vettem be, hanem általános keresésekre, technikák összehasonlítására, dokumentációk gyors feltárására és egy-egy megoldási irány előzetes felmérésére. Fejlesztési célra ezen kívül más MI-eszközt nem alkalmaztam.

10.2. Felhasználási területek

Az MI használata a projektben elsősorban kódgenerálásra épült. A fejlesztés során a kód jelentős részének első változatát MI segítségével állítottam elő. Ez különösen igaz volt a backend route-ok, szolgáltatási rétegek, validációs logikák, segédfüggvények és a Supabase-hez kapcsolódó SQL lekérdezések, illetve RPC-függvények megírására. A Supabase SQL-réteg kialakításában kifejezetten sokat támaszkodtam az MI-re, mert ezen a területen gyorsan tudott olyan induló megoldásokat adni, amelyeket ezután a saját igényeimhez igazítottam.

Az automatizált tesztelés területén még hangsúlyosabb volt a szerepe. A pytest-alapú tesztkészlet első verzióit gyakorlatilag teljes egészében MI-vel generáltattam. Ez nem azt jelentette, hogy a teszteket változtatás nélkül vettem át, hanem azt, hogy a tesztvázak, a fixture-ök, a tipikus happy path és hibakezelési esetek nagy részét az MI készítette el, én pedig ezeket felülvizsgáltam, pontosítottam és a projekt saját viselkedéséhez igazítottam.

Az MI-t emellett tervezési előkészítésre is használtam. A tipikus munkamenetem az volt, hogy először leírtam, mit szeretnék funkcionálisan elérni, milyen kényszerekkel és milyen architektúráis elvárásokkal. Erre az MI általában egy implementációs tervvel vagy részletes megoldási javaslattal válaszolt. Ezt a tervet ritkán fogadtam el elsőre: pontosítottam a határokat, szűkítettem a feladatot, tisztáztam az adatfolyamot vagy a kívánt viselkedést, majd csak ezután generáltattam le a konkrét kódot. A rendszer tehát nem önálló tervezőként működött, hanem olyan eszközként, amely gyorsan reagált a fokozatosan pontosított specifikációra.

10.3. Validáció és kritikus szemlélet

Az MI kimenetét a fejlesztés során nem kész megoldásként, hanem kiindulási javaslatként kezeltem. A legfontosabb ellenőrzési mechanizmusom az volt, hogy a kívánt működést először én határoztam meg, és csak ezután kértem implementációt. A gyakorlatban ez azt jelentette, hogy a fejlesztési ciklus több lépésből állt: cél meghatározása, az MI által javasolt terv átnézése, a terv pontosítása, a kód generálása, végül manuális és automatizált ellenőrzés. Ez a folyamat különösen fontos volt azért, mert a modellek sokszor technikailag plauzibilis, de szemléletében vagy rendszerillesztésében nem megfelelő megoldást adtak.

Többször előfordult, hogy az MI által generált kód nem azért volt problémás, mert szintaktikailag hibás lett volna, hanem mert nem azt az architektúráis vagy üzleti logikát valósította meg, amit én szerettem volna. Ennek tipikus példája a Supabase SQL-réteg volt: az MI gyakran gyorsan előállított egy látszólag korrekt lekérdezést vagy RPC-függvényt, de a lekérdezés szűrése, visszatérési szerkezete vagy a projekt többi részéhez való illeszkedése módosítást igényelt. Ilyenkor nem újragondolás nélkül fogadtam el a javaslatot, hanem addig pontosítottam a követelményeket, amíg a generált megoldás a kívánt működést nem követte.

Hasonló tapasztalatom volt a tesztekkel is. Az MI nagyon gyorsan tudott nagy mennyiségű tesztkódot előállítani, de ezek első változatban gyakran túlzottan a tipikus sikeres lefutásokra koncentráltak. Nekem kellett eldöntenem, hogy melyek azok a hibás bemenetek, szélső esetek és jogosultsági helyzetek, amelyeket valóban le kell fedni. A validációt ezért nem pusztán a generált szöveg vagy kód minősége jelentette, hanem az, hogy a végeredmény megfelelt-e a saját elképzelésemnek, jól illeszkedett-e a meglévő rendszerbe, és a tényleges tesztelés során is helyesnek bizonyult-e.

10.4. Ahol nem használtam MI-t

Tudatosan nem bíztam az MI-re az alapvető architekturális és technológiai döntéseket. A fő rendszerfelépítés, a frontend-backend felosztás, a valós idejű kommunikáció módja, a Supabase használata, a kreditrendszer helye a rendszerben, valamint a projekt általános technológiai iránya a saját döntéseim eredménye volt. Az MI ezen a ponton legfeljebb alternatívákat vagy ellenőrző szempontokat tudott adni, de a végső döntést minden esetben én hoztam meg.

Nem használtam MI-t a technikák tényleges kipróbálására és ellenőrzésére sem. Az, hogy egy adott megközelítés a projektben valóban működik-e, milyen korlátai vannak, hogyan viselkedik integrált környezetben, és megfelel-e az elvárt felhasználói élménynek, minden esetben saját teszteléssel dőlt el. Ugyanez igaz volt azokra a helyzetekre is, amikor a rendszer viselkedésének végső formáját kellett meghatároznom: az MI tudott javaslatot tenni, de azt, hogy pontosan mit és hogyan szeretnék a programban megvalósítani, nekem kellett definiálnom.

Szintén nem engedtem át az MI-nek a projekt határainak kijelölését. A feladatok priorizálása, a funkciók szűkítése, az implementációk végső egyszerűsítése, valamint annak eldöntése, hogy egy adott megoldás túl bonyolult-e a projekt céljaihoz képest, mérnöki döntés maradt. Ezekben a helyzetekben az MI sokszor inkább a túltermelés irányába vitte volna a megoldást, nekem kellett visszahozni a rendszert egy kezelhető, fenntartható formába.

10.5. Tanulságok

A projekt legfontosabb tanulsága számomra az volt, hogy az MI a legnagyobb értéket akkor adja, ha a probléma már kellően pontosan meg van fogalmazva. Ilyenkor kifejezetten nagy gyorsulást jelent: rövid idő alatt képes nagy mennyiségű kódot, SQL-t vagy tesztet előállítani, és jelentősen csökkenti a repetitív munka mennyiségét. Ez különösen igaz volt a tesztkészlet bővítésére, a Supabase SQL-réteg kialakítására és az ismétlődő backend logikák első implementációjára.

Ezzel szemben azt is megtanultam, hogy az MI gyenge pontja nem feltétlenül a szintaxis, hanem a szándék pontos követése. Sok esetben nem hibás, hanem „majdnem jó” megoldást adott: olyat, amely első olvasásra meggyőzőnek tűnt, de nem pontosan azt valósította meg, amit a projekt logikája vagy az én szemléletem megkívánt. Emiatt a sikeres használat kulcsa nem pusztán a jó prompt, hanem a fokozatos pontosítás, a szűk feladatokra bontás és a következetes ellenőrzés volt.

A folyamat összességében megerősítette bennem, hogy a mesterséges intelligencia fejlesztési környezetben rendkívül hatékony gyorsítóeszköz, de nem helyettesíti a mérnöki felelősséget. A következő hasonló projektben még tudatosabban bontanám kisebb egységekre a feladatokat, és még korábban rögzíteném az elfogadási feltételeket, mielőtt kódot generáltatnék. Minél pontosabban tudtam meghatározni a kívánt működést, annál hasznosabb volt az MI; ahol viszont a cél homályos maradt, ott a generált kód is nagyobb utólagos korrekciót igényelt.

11. Összefoglalás

11.1. Eredmények

Szakedolgozatomban bemutattam az InstaCover webalkalmazást, egy AI-alapú könyvborító-generátort indie szerzők számára. Az alkalmazás sikeresen egyesíti a nagy nyelvi modelleket, a képgenerálási technológiát és a szerver oldali renderelést egy egységes, felhasználóbarát felületen.

A főbb megvalósított képességek:

- Háromágú generálási pipeline standard, stílusreferencia és sablon módokkal, amelyek eltérő felhasználói igényeket szolgálnak ki 2-5 lépéses, automatikus folyamatokkal
- Stílusreferencia rendszer, amely feltöltött borítók AI-alapú dekompozícióját és a stílus szelektív reprodukcióját végzi (vizuális, tipográfiai, vagy mindkettő)
- WebSocket-alapú valós idejű kommunikáció lépésenkénti visszajelzéssel, állapot-visszaállítással és megszakítási lehetőséggel
- Többretegű biztonsági modell Row-Level Security-vel, prompt injection detektálással, atomikus kredit-műveletekkel és rate limitinggel
- Automatizált minőségbiztosítás 204 backend teszttel, CI/CD pipeline-nal és TypeScript típusellenőrzéssel

A zárt béta tesztelés során a rendszer stabilan működött, és a pipeline-ok a különböző konfigurációkban sikeresen produkáltak borítókat.

11.2. Tapasztalatok

A fejlesztés során több fontos tanulságot szűrtem le:

A prompt engineering iteratív folyamat. A generálási minőség legnagyobb mértékben a prompt szövegtől függ. Az externalizált prompts.json lehetővé tette, hogy a promptokat kódváltoztatás nélkül finomítsam, ami kulcsfontosságú volt az iteratív javításhoz.

Az aszinkron job-kezelés komplexitást ad. A WaveSpeed API aszinkron modellje (submit → poll) robusztus hibakezelést igényel: timeout, retry, stale detection. A polling mechanizmus egyszerűbb egy message queue-nál, de figyelni kell a végtelen ciklus és erőforráspazarlás kockázatára.

A lépésenkénti kredit-levonás méltányosabb. Az előre levonás + visszatérítés modellel szemben a lépésenkénti levonás biztosítja, hogy sikertelen generálás esetén a felhasználó csak a ténylegesen elhasznált erőforrásokért fizet.

Az AI-eszközök hatékony kiegészítők, de nem helyettesítők. A fejlesztés során szisztematikusan alkalmaztam nagy nyelvi modelleket kódgenerálásra, tesztírásra és tervezési előkészítésre. A tapasztalat egyértelműen megmutatta, hogy ezek az eszközök valódi sebességnövelést hoznak jól behatárolt feladatoknál, ugyanakkor a mérnöki ítélőképességet nem pótolják: a generált kód validálása és a kritikus döntések meghozatala mindig emberi felelősség maradt. A részletes tapasztalatokat a 10. fejezet tartalmazza.

11.3. Továbbfejlesztési lehetőségek

A rendszer több irányban bővíthető:

- Stripe vagy PayPal fizetési integráció, hogy a felhasználók önállóan vásárolhassanak krediteket (jelenleg admin által kiosztott)
- Frontend tesztek Vitest-tel és React Testing Library-vel a kritikus komponensekhez, Playwright E2E tesztek a fő felhasználói folyamatokhoz
- LoRA finomhangolás egyéni stílusmodellek tanítására specifikus vizuális stílusokhoz, a stílusreferencia rendszer bővítéseként
- Migrálás alternatív objektumtárolásra (Cloudflare R2 vagy AWS S3) a Supabase free tier tárhelykorlátainak megkerülésére
- Szerver oldali események a polling helyett a WaveSpeed eredményeknél, ha az API támogatná

11.4. Záró gondolatok

Az InstaCover demonstrálja, hogy a modern AI szolgáltatások (LLM + képgenerálás) megfelelő orkesztrációval képesek egy korábban kizárólag emberi kreativitást igénylő feladatot, a könyvborító-tervezést, hozzáférhetővé tenni azok számára is, akik nem rendelkeznek grafikai ismeretekkel. A felhasználónak elegendő leírnia könyvét, és a rendszer percekben belül professzionális minőségű borítót ad.

Irodalomjegyzék

- [1] React: A JavaScript library for building user interfaces. <https://react.dev> (Utolsó megtekintés: 2025. 12. 15.)
- [2] TypeScript: Typed JavaScript at Any Scale. <https://www.typescriptlang.org> (Utolsó megtekintés: 2025. 12. 15.)
- [3] Vite: Next Generation Frontend Tooling. <https://vite.dev> (Utolsó megtekintés: 2025. 12. 15.)
- [4] Flask: A lightweight Python web framework. <https://flask.palletsprojects.com> (Utolsó megtekintés: 2025. 12. 15.)
- [5] Supabase: The open source Firebase alternative. <https://supabase.com/docs> (Utolsó megtekintés: 2025. 12. 15.)
- [6] Socket.IO: Bidirectional and low-latency communication. <https://socket.io/docs> (Utolsó megtekintés: 2025. 12. 15.)
- [7] TailwindCSS: A utility-first CSS framework. <https://tailwindcss.com/docs> (Utolsó megtekintés: 2025. 12. 15.)
- [8] OpenRouter: A unified interface for LLM APIs. <https://openrouter.ai/docs> (Utolsó megtekintés: 2025. 12. 15.)
- [9] WaveSpeed AI: Fast image generation API. <https://wavespeed.ai/docs> (Utolsó megtekintés: 2025. 12. 15.)
- [10] Docker: Develop, Ship, and Run applications in containers. <https://docs.docker.com> (Utolsó megtekintés: 2025. 12. 15.)
- [11] GitHub Actions: Automate your workflow. <https://docs.github.com/en/actions> (Utolsó megtekintés: 2025. 12. 15.)
- [12] Canva: AI Book Cover Generator. <https://www.canva.com/ai-book-cover-generator/> (Utolsó megtekintés: 2025. 10. 04.)
- [13] GetCovers: Custom Book Cover Design. <https://getcovers.com> (Utolsó megtekintés: 2025. 10. 04.)
- [14] TheBookCoverDesigner: Premade Book Covers. <https://thebookcoverdesigner.com> (Utolsó megtekintés: 2025. 10. 04.)
- [15] PostgreSQL Row Level Security. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (Utolsó megtekintés: 2025. 12. 15.)
- [16] WebSocket Protocol (RFC 6455). https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API (Utolsó megtekintés: 2025. 12. 15.)
- [17] OWASP: Prompt Injection. <https://owasp.org/www-project-top-10-for-large-language-model-applications/> (Utolsó megtekintés: 2025. 12. 15.)
- [18] Written Word Media: Self-Publishing Statistics 2024. <https://www.writtenwordmedia.com/self-publishing-statistics/> (Utolsó megtekintés: 2025. 10. 01.)
- [19] TanStack Query: Powerful asynchronous state management. <https://tanstack.com/query> (Utolsó megtekintés: 2025. 12. 15.)
- [20] Framer Motion: A production-ready motion library for React. <https://www.framer.com/motion/> (Utolsó megtekintés: 2025. 12. 15.)

Nyilatkozat

Alulírott Kiss Norbert programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Aláírás